

리눅스 기본 명령어

LFCS 도메인 1 (배점 20%) — 로그인, 파일 조작, 검색, 텍스트 처리, 퍼미션, 아카이브, Git

목차

1. 로그인과 터미널
2. 파일시스템 트리와 경로
3. 파일·디렉터리 다루기
4. 파일 내용 보기
5. 리다이렉션과 파이프
6. 파일 검색 — find, locate
7. 텍스트 검색 — grep과 정규표현식
8. 텍스트 처리 — sed, awk, sort, cut
9. 파일 비교
10. vim 편집기
11. 퍼미션과 소유권
12. 링크 — 하드링크와 심볼릭 링크
13. 아카이브와 압축
14. Git 기초
15. 도움말 찾기
16. 실습 체크리스트 / 자주 하는 실수

1. 로그인과 터미널

1.1 용어 정리

console, virtual terminal, terminal emulator는 자주 혼용되지만 원래는 다른 물건이었습니다. 컴퓨터가 한 대뿐이던 시절, 여러 사람이 각자 화면과 키보드가 달린 *물리 장치*를 통해 그 한 대에 접속했는데 그 장치가 터미널이었습니다. 지금은 전부 소프트웨어로 구현됩니다.

용어	지금의 의미
콘솔(console)	부팅 시 텍스트가 쪽 올라가는 그 화면. 시스템에 직접 붙은 기본 출력
가상 터미널(vt)	<code>Ctrl+Alt+F2~F6</code> 으로 전환되는 텍스트 로그인 화면
터미널 에뮬레이터	GUI 환경에서 창으로 뜨는 앱 (GNOME Terminal, iTerm 등)
셸(shell)	터미널 안에서 실제로 명령을 해석하는 프로그램 (bash, zsh)

정리하면 **터미널은 입출력 창구**, **셸은 명령 해석기**입니다. 이 구분을 해주면 "터미널을 바꿨다"와 "셸을 바꿨다"가 전혀 다른 얘기라는 게 이해됩니다.

1.2 로그인 방식 네 가지

- **로컬 텍스트 모드** — 서버에 직접 연결된 콘솔. 네트워크가 죽어도 접근 가능해서 장애 대응의 최후 수단
- **로컬 그래픽 모드** — 데스크톱 환경의 로그인 화면
- **원격 텍스트 모드** — SSH. 서버 운영의 99%가 여기서 일어납니다
- **원격 그래픽 모드** — VNC, X11 포워딩, RDP

```
ssh jihoon@192.168.0.20
```

```
ssh -p 2222 jihoon@server.example.com # 포트 지정
ssh -i ~/.ssh/id_ed25519 jihoon@server # 키 파일 지정
```

클라우드 인스턴스는 대부분 비밀번호 로그인에 꺼져 있고 키 인증만 허용합니다. 키를 만들 때는 `ssh-keygen -t ed25519` 를 쓰세요. RSA보다 짧고 빠르며 보안 강도도 충분합니다.

1.3 프롬프트 읽기

```
jihoon@web01:~$
① ② ③④
```

① 사용자명 ② 호스트명 ③ 현재 디렉터리(~ 는 홈) ④ 프롬프트 기호. `$` 는 일반 사용자, `#` 는 root입니다. 문서에서 명령 앞에 붙은 `$` 나 `#` 는 같이 입력하는 게 아니라 권한 표시입니다.

2. 파일시스템 트리와 경로

2.1 단일 트리 구조

윈도우는 드라이브마다 C: , D: 로 나뉘지만, 리눅스는 루트(/) 하나에서 시작하는 단일 트리입니다. 두 번째 디스크를 붙여도 새 드라이브 문자가 생기는 게 아니라 트리의 특정 지점(/mnt/data 같은)에 마운트되어 붙습니다.

경로	용도
/	루트. 모든 것의 시작점
/home	일반 사용자 홈 디렉터리
/root	root의 홈. / 와 헷갈리지 말 것
/etc	설정 파일. 텍스트 파일이 원칙
/var	변하는 데이터 — 로그(/var/log), 스푼, 캐시
/tmp	임시 파일. 재부팅 시 삭제됨
/usr	사용자 프로그램과 라이브러리
/opt	서드파티·자체 설치 소프트웨어
/dev	장치 파일 (/dev/sda , /dev/null)
/proc , /sys	커널이 만드는 가상 파일시스템. 디스크에 없음
/boot	커널 이미지와 부트로더
/srv	서비스가 제공하는 데이터

2.2 절대 경로와 상대 경로

	절대 경로	상대 경로
시작	/ 로 시작	/ 로 시작하지 않음
기준	루트	현재 디렉터리
예	/var/log/syslog	log/syslog , ../etc
특징	어디서 실행해도 같은 곳	현재 위치에 따라 달라짐

```
.      # 현재 디렉터리
..     # 상위 디렉터리
~      # 내 홈 디렉터리
~jihoon # jihoon의 홈 디렉터리
-      # 직전 디렉터리 (cd - 로 왔다갔다)
```

스크립트에는 **절대 경로를 쓰세요**. cron이나 systemd가 스크립트를 실행할 때의 작업 디렉터리는 여러분이 예상하는 곳이 아닙니다. 상대 경로로 짠 백업 스크립트가 엉뚱한 위치를 지우는 사고가 여기서 나옵니다.

2.3 이동과 위치 확인

```
pwd          # 현재 위치 출력
cd /var/log  # 절대 경로로 이동
cd ..        # 한 단계 위로
cd           # 인자 없으면 홈으로
cd -         # 직전 위치로 토글
```

3. 파일·디렉터리 다루기

3.1 목록 보기 — ls

```
ls          # 기본
ls -l       # 상세 (퍼미션, 소유자, 크기, 시각)
ls -a       # 숨김 파일 포함 (.으로 시작하는 파일)
ls -h       # 크기를 K/M/G로 (-L과 함께)
ls -t       # 수정 시각순 정렬
ls -r       # 역순
ls -R       # 하위 디렉터리까지 재귀
ls -d */    # 디렉터리만

ls -lhrt    # 실무 조합: 상세 + 읽기 쉬운 크기 + 오래된 것부터
```

ls -lhrt 를 외워두면 로그 디렉터리에서 **가장 최근 파일이 화면 맨 아래**에 오게 됩니다. 파일이 많을 때 스크롤하지 않고 바로 볼 수 있어 장애 대응에서 자주 씁니다.

ls-l 출력 읽기

```
-rw-r--r--  1 jihoon devops  4096 Sep  2 10:30 report.txt
drwxr-xr-x  2 jihoon devops  4096 Sep  2 10:31 logs/
①          ② ③          ④          ⑤          ⑥          ⑦
```

① 파일 타입 + 퍼미션 ② 하드링크 수 ③ 소유자 ④ 그룹 ⑤ 크기(바이트) ⑥ 수정 시각 ⑦ 이름

맨 앞 한 글자가 타입입니다: **-** 일반 파일, **d** 디렉터리, **l** 심볼릭 링크, **b** 블록 장치, **c** 문자 장치.

3.2 생성

```
touch report.txt    # 빈 파일 생성 (있으면 시각만 갱신)
mkdir logs          # 디렉터리 생성
mkdir -p a/b/c       # 중간 경로까지 한 번에
mkdir -p /srv/{web,db,cache} # 중괄호 확장으로 여러 개
```

3.3 복사 — cp

```
cp a.txt b.txt      # 파일 복사
cp a.txt /tmp/       # 디렉터리로 복사
cp -r logs/ /backup/ # 디렉터리는 -r 필수
cp -p a.txt /backup/ # 퍼미션·시각 보존
cp -a logs/ /backup/ # 아카이브 모드 (-r + 모든 속성 보존)
cp -i a.txt b.txt    # 덮어쓰기 전 확인
cp -v a.txt b.txt    # 진행 상황 출력
```

cp -a 가 백업 용도의 정답입니다. 심볼릭 링크를 링크 그대로 유지하고 퍼미션·타임스탬프·소유권을 모두 보존합니다. 그냥 cp -r 만 쓰면 링크가 실제 파일로 복사되고 퍼미션이 바뀔 수 있습니다.

슬래시 하나의 차이

```
cp -r source /dest/ # /dest/source/ 가 생김
cp -r source/ /dest/ # 내용물이 /dest/ 아래 직접 들어감
```

rsync 에서도 같은 규칙이 적용되며, 백업 스크립트에서 디렉터리가 한 겹 더 생기는 사고의 원인입니다.

3.4 이동·이름 변경 — mv

```
mv old.txt new.txt      # 이름 변경
mv report.txt /archive/ # 이동
mv -i a.txt /tmp/       # 덮어쓰기 확인
mv -n a.txt /tmp/       # 있으면 덮어쓰지 않음
```

같은 파일시스템 안에서는 실제 데이터를 옮기지 않고 이름만 바꾸므로 즉시 끝납니다. 다른 파일시스템으로 옮기면 복사 후 삭제라 느립니다.

3.5 삭제 — rm

```
rm file.txt
rm -r logs/      # 디렉터리
rm -f file.txt   # 확인 없이 강제
rm -i *.log      # 하나씩 확인
rmdir empty/     # 빈 디렉터리만 삭제
```

리눅스에는 휴지통이 없습니다. `rm` 은 즉시 지웁니다.

특히 위험한 패턴:

- `rm -rf /` — 시스템 전체 (요즘은 `--no-preserve-root` 없이는 막히지만 방심 금물)
- `rm -rf $DIR/*` — `$DIR` 이 비어 있으면 `rm -rf /*` 가 됩니다. 스크립트에서 변수를 쓸 땐 "`${DIR:?}`" 처럼 미설정 시 중단하도록 방어하세요
- `rm -rf logs /` — 실수로 들어간 공백 하나

습관: 지우기 전에 같은 인자로 `ls` 를 먼저 실행해 무엇이 지워질지 확인하세요.

4. 파일 내용 보기

명령	용도
<code>cat file</code>	전체 출력. 짧은 파일용
<code>less file</code>	페이지 단위 열람. 큰 파일은 이걸 쓸 것
<code>head -n 20 file</code>	앞 20줄
<code>tail -n 20 file</code>	뒤 20줄
<code>tail -f file</code>	실시간 추적. 로그 볼 때 필수
<code>wc -l file</code>	줄 수 세기
<code>file target</code>	파일 종류 판별

less 안에서 쓰는 키

Space / b	다음/이전 페이지	g / G	맨 앞/맨 끝
/패턴	아래로 검색	n / N	다음/이전 결과
F	tail -f 처럼 추적	q	종료

`cat` 으로 거대한 로그 파일을 열면 화면이 폭주하고 터미널이 멈춥니다. 모르는 파일은 일단 `less` 로 여는 습관을 들이세요. `tail -f /var/log/syslog` 는 서비스를 재시작하면서 로그를 실시간으로 보는 표준 동작입니다.

5. 리다이렉션과 파이프

5.1 표준 스트림

모든 프로세스는 세 개의 기본 통로를 가집니다.

번호	이름	기본 연결
0	stdin (표준 입력)	키보드
1	stdout (표준 출력)	화면
2	stderr (표준 오류)	화면

stdout과 stderr이 분리된 이유가 중요합니다. 정상 출력은 파일로 저장하면서 오류는 화면에서 바로 보고 싶은 경우가 있기 때문입니다.

5.2 리다이렉션

```
ls > list.txt          # stdout을 파일로 (덮어쓰기)
ls >> list.txt         # 이어쓰기
ls 2> error.txt        # stderr만 파일로
ls > out.txt 2>&1      # stdout과 stderr 모두 같은 파일로
ls &> out.txt          # 위와 동일 (bash 축약형)
ls 2> /dev/null        # 오류 버리기
sort < input.txt       # 파일을 stdin으로
```

2>&1 의 순서가 중요합니다. `cmd > out.txt 2>&1` 은 정상 동작하지만 `cmd 2>&1 > out.txt` 는 stderr이 화면에 남습니다. 리다이렉션은 왼쪽부터 순서대로 적용되기 때문입니다. 후자는 "stderr을 (현재의) stdout인 화면으로 보낸 다음, stdout만 파일로 바꾼다"는 뜻이 됩니다.

`/dev/null` 은 무엇을 써도 사라지는 특수 장치입니다. cron 작업에서 불필요한 출력이 메일로 날아오는 걸 막을 때 `>/dev/null 2>&1` 을 붙입니다.

5.3 파이프

앞 명령의 stdout을 다음 명령의 stdin으로 연결합니다. 리눅스 명령어의 진짜 힘이 여기 있습니다.

```
ps aux | grep nginx
cat access.log | grep " 500 " | wc -l
ls -l | sort -k5 -n | tail -5      # 가장 큰 파일 5개
history | grep ssh | less
```

tee — 저장하면서 화면에도 출력

```
make 2>&1 | tee build.log
echo "text" | sudo tee /etc/config  # sudo로 파일 쓰기
```

`sudo echo "x" > /etc/file` 은 **동작하지 않습니다**. 리다이렉션은 sudo가 아니라 **현재 셸**이 처리하므로 권한이 없어 실패합니다. `echo "x" | sudo tee /etc/file` 이 정답이고, 이어쓰기는 `tee -a` 입니다. 자주 나오는 함정입니다.

5.4 xargs

파이프로 넘어온 내용을 *인자*로 바꿔 넘깁니다. stdin을 못 받는 명령에 필요합니다.

```
find . -name "*.tmp" | xargs rm
find . -name "*.log" -print0 | xargs -0 rm  # 공백 포함 파일명 안전 처리
```

6. 파일 검색 — find, locate

6.1 find — 실시간 탐색

```
find /var/log -name "*.log"      # 이름
find . -iname "readme*"         # 대소문자 무시
find / -type d -name "conf"     # 디렉터리만
find /home -user jihoon         # 소유자
find / -size +100M              # 100MB 초과
find /var/log -mtime -7         # 7일 이내 수정
find /var/log -mtime +30        # 30일보다 오래됨
find / -perm -4000 2>/dev/null  # SUID 파일 (보안 점검)
```

찾은 결과로 작업하기

```
find /tmp -name "*.tmp" -delete
find /var/log -name "*.log" -mtime +30 -exec gzip {} \;
find . -name "*.conf" -exec cp {} /backup/ \;
```

{ } 는 찾은 각 파일로 치환되고, \; 가 명령의 끝입니다. \; 대신 + 를 쓰면 여러 파일을 한 번에 넘겨 훨씬 빠릅니다.

6.2 locate — 인덱스 검색

```
locate nginx.conf
sudo updatedb      # 인덱스 갱신
```

미리 만들어둔 DB를 조회하므로 즉시 나오지만, **방금 만든 파일은 못 찾습니다**. 빠른 위치 확인은 locate, 정확한 조건 검색은 find로 나눠 쓰면 됩니다.

6.3 실행 파일 위치

```
which python3      # PATH에서 찾은 실행 파일 경로
whereis python3     # 실행 파일 + man + 소스
type ll             # 별칭/함수/내장 명령 여부까지 판별
```

7. 텍스트 검색 — grep과 정규표현식

7.1 grep 기본

```
grep "error" app.log
grep -i "error" app.log      # 대소문자 무시
grep -r "TODO" ./src        # 디렉터리 재귀
grep -n "error" app.log      # 줄 번호 표시
grep -v "debug" app.log      # 매칭 안 되는 줄만 (제외)
grep -c "error" app.log      # 개수만
grep -l "error" *.log        # 파일명만
grep -w "test" file          # 단어 단위 (testing 제외)
grep -A3 -B3 "error" app.log # 앞뒤 3줄 함께
```

장애 분석의 기본기는 `-i` (대소문자 무시)와 `-A/-B` (전후 문맥)입니다. 로그의 에러 표기는 `ERROR`, `Error`, `error` 가 섞여 있고, 에러 메시지 자체보다 직전 몇 줄에 원인이 있는 경우가 많습니다.

7.2 정규표현식 기본

기호	의미	예
<code>.</code>	임의의 한 글자	<code>a.c</code> → <code>abc</code> , <code>a1c</code>
<code>*</code>	앞 문자 0회 이상	<code>ab*c</code> → <code>ac</code> , <code>abc</code> , <code>abbc</code>
<code>^</code>	줄 시작	<code>^Error</code>
<code>\$</code>	줄 끝	<code>fail\$</code>
<code>[abc]</code>	집합 중 하나	<code>[Ee]rror</code>
<code>[a-z]</code>	범위	<code>[0-9]</code>
<code>[^abc]</code>	집합에 없는 것	<code>[^0-9]</code>
<code>\</code>	이스케이프	<code>\.</code> → 실제 점

대괄호 안의 `^` 는 부정이고, 대괄호 밖의 `^` 는 줄 시작입니다. 같은 기호가 위치에 따라 뜻이 달라집니다.

7.3 확장 정규표현식 (ERE)

`grep -E` 또는 `egrep` 을 쓰면 추가 기호를 백슬래시 없이 쓸 수 있습니다.

기호	의미	예
<code>+</code>	1회 이상	<code>[0-9]+</code>
<code>?</code>	0회 또는 1회	<code>colou?r</code>
<code>{n,m}</code>	n~m회	<code>[0-9]{1,3}</code>
<code> </code>	또는	<code>error fail</code>
<code>()</code>	그룹	<code>(ab)+</code>

실전 예

```
# IP 주소 형태 찾기
grep -E "[0-9]{1,3}(\.[0-9]{1,3}){3}" access.log
```



```
# 5xx 응답 코드만
grep -E " 5[0-9]{2} " access.log

# 빈 줄과 주석을 제외한 실제 설정만 보기 (매우 자주 씀)
grep -Ev "^s*(#|$)" /etc/ssh/sshd_config
```

마지막 명령은 실무에서 정말 자주 씁니다. 설정 파일 대부분이 주석으로 뒤덮여 있어서, **실제로 활성화된 설정만** 뽑아보면 현재 상태를 한눈에 파악할 수 있습니다. 별칭으로 등록해주세요.

7.4 문자 클래스

```
[:digit:] 숫자      [:alpha:] 영문자
[:alnum:] 영숫자    [:space:] 공백/탭
[:upper:] 대문자    [:lower:] 소문자
```

로케일에 안전하다는 장점이 있습니다. [a-z] 는 로케일에 따라 예상과 다르게 동작할 수 있습니다.

8. 텍스트 처리 — sed, awk, sort, cut

8.1 sed — 치환과 편집

```
sed 's/old/new/' file      # 각 줄의 첫 번째만
sed 's/old/new/g' file     # 모두
sed 's/old/new/gi' file    # 대소문자 무시
sed -i 's/old/new/g' file  # 파일 직접 수정
sed -i.bak 's/old/new/g' file # 백업 남기고 수정
sed -n '10,20p' file       # 10~20줄만 출력
sed '/^#/d' file           # 주석 줄 삭제
sed '/^$/d' file           # 빈 줄 삭제
```

sed -i 는 되돌릴 수 없습니다. 설정 파일에 쓸 때는 반드시 -i.bak 으로 백업을 남기거나, -i 없이 먼저 실행해 출력 결과를 눈으로 확인하세요.

8.2 awk — 열 단위 처리

공백으로 나뉜 열을 \$1 , \$2 ...로 다룹니다. 로그 분석의 주력 도구입니다.

```
awk '{print $1}' access.log      # 첫 번째 열
awk '{print $1, $7}' access.log  # 여러 열
awk -F: '{print $1}' /etc/passwd # 구분자 지정
awk '$9 == 500 {print $7}' access.log # 조건
awk '{sum += $5} END {print sum}' file # 합계
awk 'NR > 1' file                # 헤더 제외
```

실전 조합 — 접속 IP 상위 10개

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

이 한 줄이 파이프의 위력을 잘 보여줍니다. IP 추출 → 정렬 → 중복 세기 → 개수 역순 정렬 → 상위 10개. `uniq -c` 는 **정렬된 입력**에서만 제대로 동작하므로 앞의 `sort` 가 반드시 필요합니다.

8.3 sort, uniq, cut

```
sort file      # 사전순
sort -n file   # 숫자순
sort -r file   # 역순
```

```
sort -k3 -n file      # 3번째 열 기준 숫자 정렬
sort -u file          # 정렬 + 중복 제거

uniq file             # 연속 중복 제거
uniq -c file          # 개수 세기
uniq -d file          # 중복된 것만

cut -d: -f1 /etc/passwd # : 구분, 1번 필드
cut -c1-10 file       # 1~10번째 글자

tr 'a-z' 'A-Z' < file # 대문자 변환
tr -d ' ' < file      # 공백 제거
```

9. 파일 비교

```
diff a.txt b.txt          # 차이 표시
diff -u a.txt b.txt       # 통합 형식 (Git과 같은 형식)
diff -y a.txt b.txt       # 좌우 나란히. 차이는 | 로 표시
diff -r dir1/ dir2/       # 디렉터리 비교
diff -q a.txt b.txt       # 다른지만 확인

cmp a.bin b.bin           # 바이너리 비교. 첫 차이 위치 보고
md5sum a.iso b.iso        # 해시로 동일성 확인
sha256sum download.iso    # 다운로드 무결성 검증
```

diff -u 출력에서 - 는 원본에만 있는 줄, + 는 새 파일에만 있는 줄입니다. diff 기본 출력에서는 < 와 > 로 표시됩니다.

설정 파일을 수정하기 전 cp nginx.conf nginx.conf.bak 으로 백업해두면, 나중에 diff -u nginx.conf.bak nginx.conf 로 내가 정확히 뭘 바꿨는지를 확인할 수 있습니다. 장애 시 롤백 판단에 결정적입니다.

10. vim 편집기

10.1 모드 개념

vim이 어려운 이유는 **모드** 때문입니다. 이것만 이해하면 나머지는 암기입니다.

모드	진입	하는 일
일반(Normal)	Esc	이동, 삭제, 복사. 시작 시 기본 모드
입력(Insert)	i , a , o	글자 입력
비주얼(Visual)	v , V	영역 선택
명령(Command)	:	저장, 종료, 치환

vim을 열자마자 타이핑했는데 글자가 안 들어가고 이상한 일이 벌어지는 이유는, 시작이 **일반 모드**라 각 글자가 명령으로 해석되기 때문입니다. Esc 를 누르고 u (실행 취소)를 반복하면 복구됩니다.

10.2 필수 키

```
--- 입력 모드 진입 ---
i   커서 앞부터 입력      a   커서 뒤부터 입력
I   줄 맨 앞부터          A   줄 맨 뒤부터
o   아래에 새 줄          O   위에 새 줄

--- 이동 (일반 모드) ---
h j k l   좌 하 상 우      0 / $   줄 처음 / 끝
gg / G    문서 처음 / 끝   :42    42번 줄로
w / b     다음 / 이전 단어

--- 편집 ---
x   한 글자 삭제          dd   한 줄 삭제(잘라내기)
5dd  다섯 줄 삭제        yy   한 줄 복사
p   붙여넣기              u   실행 취소
Ctrl+r 재실행

--- 검색·치환 ---
/패턴      아래로 검색 (n: 다음, N: 이전)
:%s/old/new/g   전체 치환
:%s/old/new/gc  전체 치환 + 하나씩 확인
```

--- 저장·종료 ---

```
:w    저장          :q    종료
:wq   저장 후 종료   :q!   저장 안 하고 강제 종료
:x    변경 시에만 저장 후 종료
```

최소 생존 세트: i 로 입력력 → Esc → :wq 로 저장 종료. 잘못 건드렸으면 :q! 로 탈출. 이 네 개만 알아도 설정 파일 편집은 가능합니다. LFCS 시험은 GUI 가 없으므로 vim은 선택이 아니라 필수입니다.

10.3 읽기 전용으로 열린 경우

권한 없는 파일을 열고 편집한 뒤 :wq 하면 E45: 'readonly' option is set 이 뜹니다. 이때 vim을 종료하지 않고 저장하는 방법이 있습니다.

```
:w !sudo tee %
```

다만 근본 해결은 처음부터 `sudo vim` 또는 `sudoedit` 으로 여는 것입니다.

11. 퍼미션과 소유권

11.1 세 주체 × 세 권한

```
-rwxr-xr--
|  | |  |  |
소유자 그룹 기타
```

기호	숫자	파일에서	디렉터리에서
r	4	내용 읽기	목록 보기 (ls)
w	2	내용 수정	파일 생성 · 삭제 · 이름변경
x	1	실행	진입 (cd)과 내부 접근

디렉터리의 **x** 가 핵심입니다. **x** 없이 **r** 만 있으면 파일 이름 목록은 볼 수 있어도 그 안으로 들어가거나 파일 내용을 읽을 수 없습니다. 반대로 **x** 만 있으면 목록은 못 보지만 *정확한 파일명을 알고 있다면* 접근할 수 있습니다.

또 하나 — **파일 삭제 권한은 파일이 아니라 그 파일이 들어 있는 디렉터리의 w** 에 달려 있습니다. 읽기 전용 파일도 디렉터리 쓰기 권한이 있으면 지워집니다.

또 하나 — 파일 삭제 권한은 파일이 아니라 그 파일이 들어 있는 디렉터리의 **w** 에 달려 있습니다. 읽기 전용 파일도 디렉터리 쓰기 권한이 있으면 지워집니다.

11.2 chmod

```
--- 숫자 방식 ---
chmod 755 script.sh      # rwxr-xr-x
chmod 644 file.txt        # rw-r--r--
chmod 600 ~/.ssh/id_ed25519 # rw----- (개인키 필수)
chmod 700 ~/.ssh/          # drwx-----

--- 기호 방식 ---
chmod u+x script.sh      # 소유자에게 실행 권한 추가
chmod g-w file            # 그룹의 쓰기 제거
chmod o= file             # 기타 사용자 권한 전부 제거
chmod a+r file            # 모두에게 읽기
chmod -R 755 dir/         # 재귀
```

자주 쓰는 값

값	의미	용도
644	rw-r--r--	일반 파일 기본
755	rwxf-rf-x	스크립트, 디렉터리 기본
600	rw-----	SSH 개인키, 비밀 설정
700	rwxf-----	개인 전용 디렉터리
664	rw-rw-r--	그룹 협업 파일
2775	rwxfwsf-x	setgid 협업 디렉터리

SSH 개인키의 퍼미션이 600 보다 느슨하면 SSH가 **키 사용을 거부**합니다. `Permissions 0644 for 'id_ed25519' are too open` 오류가 그것입니다. `chmod 600` 으로 고치면 됩니다.

반대로 `chmod -R 777` 은 절대 습관화하지 마세요. "권한 문제가 나면 777"은 문제를 해결하는 게 아니라 보안을 포기하는 것입니다. 대부분 소유권(`chown`) 문제입니다.

11.3 chown, chgrp

```
sudo chown jihoon file.txt
sudo chown jihoon:devops file.txt # 소유자:그룹 동시
sudo chown -R jihoon:devops /srv/app
sudo chgrp devops file.txt # 그룹만
```

11.4 특수 퍼미션

비트	값	효과
SUID	4000	실행 시 파일 소유자 권한으로 동작 (passwd 가 대표 예)
SGID	2000	디렉터리에 걸면 새 파일이 디렉터리 그룹 을 상속
Sticky	1000	디렉터리 안에서 자기 파일만 삭제 가능 (/tmp)

```
chmod 2775 /srv/shared # SGID — 협업 디렉터리
chmod 1777 /tmp # Sticky — 공용 임시 디렉터리
find / -perm -4000 2>/dev/null # SUID 파일 감사
```

11.5 umask

새로 만들어지는 파일의 기본 퍼미션을 결정합니다. **빼는 값**입니다.

```
umask # 현재 값 확인 (보통 0022)
umask 002 # 그룹 쓰기 허용
```

umask	파일 (666 기준)	디렉터리 (777 기준)
022	644	755
002	664	775
077	600	700

파일이 666에서 시작하는 이유는 새 파일에 실행 권한을 자동으로 주지 않기 위해서입니다.

12. 링크 — 하드링크와 심볼릭 링크

```
ln original.txt hardlink.txt      # 하드링크
ln -s /path/to/original softlink  # 심볼릭 링크
ln -sf /new/target existing_link  # 링크 대상 교체
```

	하드링크	심볼릭 링크
실체	같은 inode를 가리키는 또 하나의 이름	경로 문자열을 담은 별도 파일
원본 삭제 시	데이터 유지 (링크가 남아 있으면)	깨짐 (dangling link)
파일시스템 경계	넘을 수 없음	넘을 수 있음
디렉터리 대상	불가	가능
ls -l 표시	일반 파일과 동일	l 로 시작, -> 표시

파일의 링크 수와 inode 번호는 이렇게 확인합니다.

```
ls -li file.txt      # 맨 앞이 inode 번호
stat file.txt
```

실무에서는 **심볼릭 링크가 압도적으로 많이** 쓰입니다. 대표 패턴이 무중단 배포입니다. /srv/app/current 를 실제 버전 디렉터리로 향하는 심볼릭 링크로 두고, 배포 시 ln -sfn 으로 대상만 바꾸면 전환이 원자적으로 끝나고 롤백도 링크만 되돌리면 됩니다.

13. 아카이브와 압축

13.1 개념 구분

아카이브는 여러 파일을 하나로 묶는 것(tar), **압축**은 크기를 줄이는 것(gzip)입니다. 리눅스에서는 이 둘이 분리되어 있고, .tar.gz 는 "묶은 뒤 압축했다"는 뜻입니다.

13.2 tar

```
tar -cvf backup.tar dir/      # 묶기 (압축 없음)
tar -czvf backup.tar.gz dir/  # 묶기 + gzip
tar -cjvf backup.tar.bz2 dir/ # 묶기 + bzip2
tar -cJvf backup.tar.xz dir/  # 묶기 + xz

tar -xzvf backup.tar.gz       # 풀기
tar -xzvf backup.tar.gz -C /tmp/ # 특정 위치에 풀기
tar -tzvf backup.tar.gz       # 내용 목록만 확인 (풀지 않음)
```

옵션	의미	암기
-c	생성	create
-x	추출	extract
-t	목록	list
-v	진행 표시	verbose
-f	파일명 지정	file — 항상 마지막에

-z	gzip	.gz
-j	bzip2	.bz2
-J	xz	.xz

-f 뒤에 바로 파일명이 와야 합니다. `tar -cfz backup.tar.gz dir/` 처럼 순서를 틀리면 `z` 라는 이름의 파일을 만들려 합니다.

그리고 모르는 아카이브는 반드시 `-t` 로 내용을 먼저 확인하세요. 절대 경로나 `../` 가 포함된 악성 아카이브가 시스템 파일을 덮어쓸 수 있습니다. 안전한 스킴은 빈 디렉터리를 만들고 그 안에서 푸는 것입니다.

13.3 개별 압축 도구

```
gzip file.txt      # file.txt.gz 생성, 원본 삭제
gunzip file.txt.gz # 해제
gzip -k file.txt   # 원본 유지
zcat file.txt.gz   # 압축 상태로 내용 보기
zgrep "error" app.log.gz # 압축 로그에서 바로 검색
```

도구	압축률	속도	용도
gzip	보통	빠름	범용, 로그 로테이션
bzip2	좋음	느림	-
xz	매우 좋음	매우 느림	배포용 아카이브
zstd	좋음	매우 빠름	최근 표준으로 부상

13.4 동기화 — rsync

```
rsync -av /src/ /dest/      # 로컬 동기화
rsync -avz /src/ user@server:/dest/ # 원격 (압축 전송)
rsync -av --delete /src/ /dest/    # 원본에 없는 건 대상에서도 삭제
rsync -av --dry-run /src/ /dest/   # 시뮬레이션 — 먼저 이걸로 확인
```

변경된 부분만 전송하므로 반복 백업에서 tar보다 훨씬 효율적입니다.

`--delete` 는 위험합니다. 경로를 잘못 지정하면 대상 디렉터리가 통째로 비워집니다. 항상 `--dry-run` 으로 먼저 확인하고, 3.3절에서 다룬 슬래시 규칙도 함께 점검하세요.

14. Git 기초

14.1 최초 설정

```
git config --global user.name "Jihoon Kim"
git config --global user.email "jihoon@example.com"
git config --list
```

14.2 기본 흐름

Git은 세 개의 영역으로 나뉩니다. 이 구조를 이해하면 명령이 자연스럽게 정리됩니다.

작업 디렉터리 --(git add)--> 스테이징 영역 --(git commit)--> 저장소

```
git init                # 새 저장소 생성
git clone <url>         # 기존 저장소 복제

git status              # 현재 상태 (가장 자주 씀)
git add file.txt        # 스테이징
git add .               # 전체 스테이징
git commit -m "메시지"  # 커밋

git log --oneline --graph # 이력 보기
git diff               # 아직 add 안 한 변경
git diff --staged      # add 했지만 commit 안 한 변경

git pull               # 원격 변경 가져오기
git push              # 원격에 반영
```

14.3 되돌리기

```
git restore file.txt      # 작업 디렉터리 변경 취소
git restore --staged file.txt # 스테이징만 취소
git commit --amend        # 직전 커밋 수정
git revert <commit>       # 되돌리는 새 커밋 생성 (안전)
git reset --hard <commit> # 이력 자체를 되돌림 (위험)
```

인프라 직무에서 Git이 필요한 이유는 코드 때문이 아니라 **설정 관리(IaC)** 때문입니다. Ansible 플레이북, Terraform 코드, Kubernetes 매니페스트, 심지어 서버 설정 파일까지 Git으로 버전 관리하는 것이 표준입니다. "누가 언제 왜 이 설정을 바꿨는가"에 답할 수 있게 됩니다.

`git push --force` 는 협업 브랜치에서 다른 사람의 커밋을 지웁니다. 꼭 필요하다면 `--force-with-lease` 를 쓰세요. 원격이 예상과 다르면 거부합니다.

15. 도움말 찾기

```
man ls                # 매뉴얼 (q로 종료, /로 검색)
man 5 passwd          # 섹션 지정 — 5번은 파일 형식
man -k network        # 키워드로 매뉴얼 검색

ls --help             # 간단한 옵션 요약
info coreutils        # GNU 상세 문서
apropos user          # man -k 와 동일

ls /usr/share/doc/    # 패키지별 문서·예제
```

man 섹션 번호

1	일반 명령	5	파일 형식·설정 파일
2	시스템 콜	7	기타 개념
3	라이브러리 함수	8	관리자 명령

man passwd 는 passwd 명령을, man 5 passwd 는 /etc/passwd 파일 형식을 보여줍니다. 설정 파일 문법이 궁금할 땐 5번 섹션을 찾으세요. man 5 crontab , man 5 fstab , man 5 limits.conf 모두 유용합니다.

LFCS는 **man 페이지 열람이 허용되는** 실기 시험입니다. 옵션을 다 외우는 것보다 필요한 정보를 man에서 빠르게 찾는 훈련이 실전에서 유리합니다.

16. 실습 체크리스트 / 자주 하는 실수

16.1 실습 체크리스트

1. 홈에서 `mkdir -p lab/{a,b,c}` 로 구조를 만들고 절대·상대 경로로 각각 이동해보기
2. `ls -lhrt /var/log` 로 최근 수정 파일 확인
3. `cp -r src /dest` 와 `cp -r src/ /dest` 의 결과 차이를 직접 비교
4. `echo "x" > /etc/test` 가 `sudo`로도 실패하는 것을 확인하고 `tee` 로 해결
5. `cmd > out 2>&1` 과 `cmd 2>&1 > out` 의 결과 차이 확인
6. `find /var/log -mtime +7 -name "*.log"` 로 오래된 로그 찾기
7. `grep -Ev "^s*(#|$)" /etc/ssh/sshd_config` 로 실제 설정만 추출
8. access 로그에서 `awk | sort | uniq -c | sort -rn | head` 로 상위 IP 집계
9. 디렉터리에서 `x` 만 제거하고 `cd` 가 막히는지 확인
10. 읽기 전용 파일을 디렉터리 쓰기 권한으로 삭제해보기 (11.1절 검증)
11. 하드링크와 심볼릭 링크를 만들고 원본을 지운 뒤 각각의 상태 비교
12. `tar -tzvf` 로 내용 확인 후 `-C` 로 지정 위치에 풀기
13. `rsync -av --dry-run` 으로 시뮬레이션 후 실제 실행
14. vim으로 파일을 열어 `:%s/old/new/gc` 치환 실습
15. `man 5 fstab` 을 열어 5번 섹션의 성격 확인

16.2 자주 하는 실수

실수	결과	올바른 방법
<code>sudo echo > /etc/file</code>	권한 오류	<code> sudo tee</code>
<code>cmd 2>&1 > file</code>	stderr이 화면에 남음	<code>cmd > file 2>&1</code>
<code>rm -rf \$DIR/*</code> (변수 미설정)	<code>/*</code> 가 삭제 대상	<code>"\${DIR:?}"</code> 로 방어
<code>tar -cfz</code> 순서 오류	엉뚱한 파일 생성	<code>-f</code> 는 항상 마지막
<code>cp -r</code> 로 백업	퍼미션·링크 손실	<code>cp -a</code> 또는 <code>rsync -a</code>
<code>uniq -c</code> 앞에 <code>sort</code> 누락	중복 집계가 틀림	<code>sort uniq -c</code>
<code>chmod -R 777</code> 로 권한 문제 해결	보안 붕괴	소유권(<code>chown</code>) 점검
SSH 개인키 644	키 사용 거부	<code>chmod 600</code>
<code>sed -i</code> 백업 없이 실행	복구 불가	<code>sed -i.bak</code>
<code>rsync --delete</code> 즉시 실행	대상 디렉터리 소실	<code>--dry-run</code> 선행
스크립트에 상대 경로	cron에서 오동작	절대 경로 사용
큰 로그를 <code>cat</code>	터미널 멈춤	<code>less</code> 또는 <code>tail</code>